WOOSH: A SOUND EFFECTS FOUNDATION MODEL

Gaëtan Hadjeres[1], Marc Ferras[1], Khaled Koutini[1], Benno Weck-Hufnagel[1*], Alexandre Bittar[1*], Thomas Hummel[1*], Zineb Lahrici[1], Hakim Missoum[1], Joan Serrà[1], and Yuki Mitsufuji[1,2]

[1] **Sony AI**
[2] **Sony Group Corporation**

Abstract

The audio research community relies on open generative models to build upon and use as baselines for novel approaches. In this report, we describe the publicly released audio generative models built at Sony AI, including architecture, training process and evaluation against other popular models. Being optimized for sound effect generation, we provide a high-quality audio encoder/decoder model, a text-audio alignment model for conditioning, and text-to-audio (T2A) and video-to-audio (V2A) generative models. Distilled T2A and V2A models for fast inference are also included in the release. Our evaluation shows competitive performance against Stable Audio Open and Tango-Flux models. Inference code and open weights are available at https://www.github.com/woosh-sfx

Contents

1 Introduction **2**

2 Woosh-AE: Audio Encoder/Decoder **3**
- 2.1 Architecture . . . 3
- 2.2 Data . . . 3
- 2.3 Training . . . 4
- 2.4 Evaluation . . . 5

3 Woosh-CLAP: Text Conditioning **6**
- 3.1 Architecture . . . 6
- 3.2 Data . . . 6
- 3.3 Training . . . 7
- 3.4 Evaluation . . . 7

4 Woosh-Flow: Text-to-Audio Generation **8**
- 4.1 Architecture . . . 8

*Work done during an internship at Sony AI.

4.2 Data . . . 8
4.3 Training . . . 9
4.4 Distillation . . . 9
4.5 Evaluation . . . 11

5 Woosh-VFlow: Video-to-Audio Generation **12**
5.1 Architecture . . . 12
5.2 Data . . . 12
5.3 Training . . . 12
5.4 Distillation . . . 13
5.5 Evaluation . . . 13

6 Conclusion **13**

A Distillation Pseudocode **17**

B Additional Results **18**

1 Introduction

Current research on generative audio modeling largely focuses on conditioned models, mainly based on textual descriptions of audio. While significant progress has been made in terms of modeling, most approaches do not provide open weights for the research community to build upon [1, 13, 14, 20]. Other approaches like [25, 31] do provide open weights, but use low audio sampling rates only up to 16 kHz. Notable exceptions are AudioLDM2 [32], StableAudio-Open (SAO) [15], and TangoFlux [20], which generate higher quality audio while addressing the generation of both general audio and music. MusicGen [7] is another high-quality open mode which exclusively specializes on music generation.

In this report, we propose a text-conditioned generative model specializing on instant high-quality sound effect generation. Based on the multimodal FLUX Kontext [27] architecture, our latent diffusion model (LDM) has been optimized from the ground up for sound effects and for professional use. As part of a public release (https://github.com/SonyResearch/woosh-sfx/), we provide inference code and open weights for non-commercial use for the full pipeline: encoder/decoder, text-conditioning, and diffusion models, the latter being further distilled for instant generation. We benchmark our public and private models against SAO, and our CLAP model against LAION-CLAP. For the best audio generation quality, a version trained exclusively on a large amount of studio-quality licensed sound effect libraries is internally available[2].

The current public release provides four models that address the text-to-audio (T2A) and video-to-audio (V2A) tasks:

- **Audio encoder/decoder (Woosh-AE)** — High-quality latent encoder/decoder providing latents for generative modeling and decoding audio from generated latents.
- **Text conditioning (Woosh-CLAP)** — Multimodal text-audio alignment model providing token latents for diffusion model conditioning.
- **T2A Generation (Woosh-Flow and Woosh-DFlow)** — Original and distilled LDMs generating audio unconditionally or from given a text prompt.
- **V2A Generation (Woosh-VFlow)** — Multimodal LDM generating audio from a video sequence with optional text prompts.

Figure 1 shows a high-level diagram of the models involved in our text-to-audio generative model.

[2]Please contact Marc Ferras or Hakim Missoum for more information about this model.

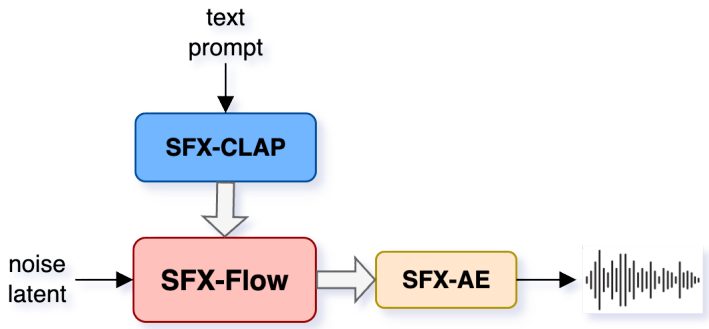

Figure 1: Inference-time layout of the Woosh models for text-to-audio generation.

2 Woosh-AE: Audio Encoder/Decoder

2.1 Architecture

The Woosh-AE encoder/decoder model is based on the VOCOS architecture [40], a GAN-based vocoder operating on the domain of short-time Fourier transform (STFT) complex coefficients. Compared to other popular encoder/decoders like Encodec [9] or DAC [26], VOCOS does not use quantization and it performs one-step down/up sampling via STFT/iSTFT efficiently. The iSTFT avoids aliasing artifacts, typically linked to transposed convolutions used in upsampling in alternative approaches [36]. Note that Woosh-AE works on monoaural audio only.

The VOCOS architecture uses a cascade of ConvNeXt blocks with residual connections both in the encoder and decoder [40], as shown in Figure 2. The last ConvNeXt block is meant to predict magnitude $\mathbf{m}$ and phase $\mathbf{p}$, from which the complex STFT is computed as $\mathbf{m}(\cos(\mathbf{p}) + j\sin(\mathbf{p})) = \mathbf{m}(\mathbf{x} + j\mathbf{y})$ prior to STFT inversion. This formulation avoids phase wrapping issues for any phase vector $\mathbf{p}$. However, our model uses a slightly different approach to computing complex STFT coefficients. The last ConvNeXt block predicts magnitude $\mathbf{m}$, and real $\mathbf{x}'$ and imaginary $\mathbf{y}'$ parts instead of phase. $\mathbf{x}'$ and $\mathbf{y}'$ are then used to compute the phase-only components on the complex plane as $\mathbf{x} = \mathbf{x}'/\mathbf{m}'$ and $\mathbf{y} = \mathbf{y}'/\mathbf{m}'$ components, with $\mathbf{m}' = \sqrt{\mathbf{x}'^2 + \mathbf{y}'^2}$. The STFT coefficients are finally obtained as $\text{softplus}(\mathbf{m})(\mathbf{x} + j\mathbf{y})$. This is an alternative way of avoiding phase wrapping issues that improved reconstruction and discrimination metrics in preliminary experiments.

2.2 Data

To train the Woosh-AE public model, we used the following datasets:

- **Freesound** — A dataset consisting of Creative Commons licensed sound clips collected on the freesound.org online platform. Only audio with public domain (CC0), CC-BY, and CC-Sampling+ was used. The dataset contains a total of 370 k audio files, with the vast majority sampled at least at 44.1 kHz. We kept the original sample descriptions from Freesound.
- **AudioCaps** — A dataset with 48 k audio clips from AudioSet [16], captioned by humans in a crowd-sourced setting [22]. The sample rate for all samples is 32 kHz.
- **WavCaps** [35] — A dataset consisting of 99 k weakly-labeled captions along with the corresponding Audioset audio clips only (WavCaps AudioSet subset). Audio samples overlapping with AudioCaps were dropped. The sample rate is 32 kHz.
- **VTCK** [45] — A dataset consisting of 44 k samples of accented English speech. Only the train split of the dataset was used, and only the audio was used, dropping any metadata or transcripts. Originally thought for high-quality speech synthesis and voice cloning, the sampling rate is 48 kHz.
- **Wapy** — An internal synthetic dataset generated using phase-modulation synthesis. For each of the 100 k audio samples, we generate a mix with 1 to 3 random audio signals, each with a random number of up to 20 sinusoidal partials, random interpolated pitch trajectories, random spectral envelopes, and random harmonic deviation. The main purpose behind this unlabeled dataset was

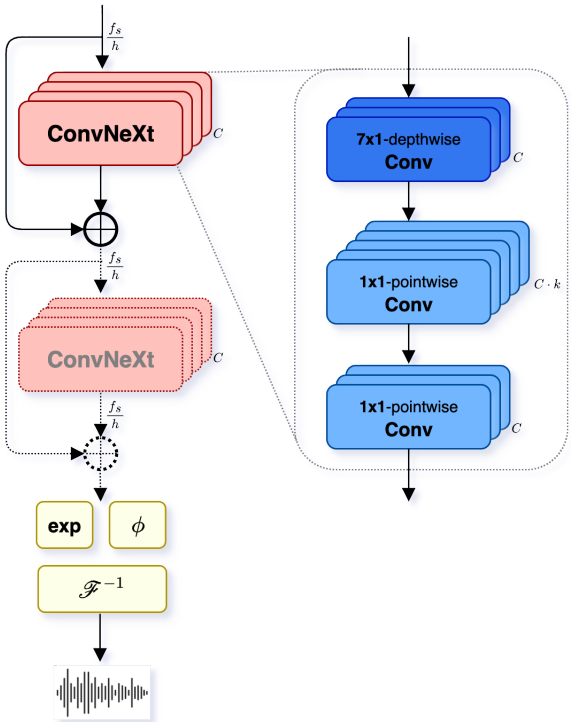

Figure 2: VOCOS decoder architecture as a cascade on ConvNeXt blocks.

to excite the entire range of frequencies during model training, with the hope of reducing audio distortion in the generated audio.

- **Internal music (IM)** — An internal dataset consisting of 78 k commercially-licensed popular music songs. Single stems as well as mixed stems were used for training, with stem labels used only for the purpose of mixing. The sample rate is 44.1 kHz.

To train the Woosh-AE private model, added to the previous list a mix of four internal studio-quality sound effect libraries, involving 1 M samples and 5500 h of audio. The pre-processing for training both public and private models consisted of resampling the audio to 48 kHz and randomly taking non-segmented 1-second long chunks from each audio.

2.3 Training

Closely following DAC [26], our model uses a GAN approach to training, minimizing a linear combination of the losses below:

- **Frequency-domain spectral loss** — To encourage modeling at multiple time scales, multi-scale L1 losses are computed on Mel spectrograms with window sizes $W = \{32, 64, 128, 256, 512, 1024, 2048\}$ and mel bin sizes $\{[5, 10, 20, 40, 80, 160, 320]\}$ were used, as in [26]. Assuming G is the encoder-decoder model, and thus the generator in the adversarial training framework:

$$\mathcal{L}_{spec} = \sum_{w \in W} \sum_{t} \|\mathcal{M}(\mathcal{S}_t(x)) - \mathcal{M}(\mathcal{S}_t(G(x)))\|_1 ,$$

  where $\mathcal{M}$ is the frequency-to-mel conversion and $\mathcal{S}_t$ is the STFT at time t.

- **Adversarial loss** — A total of 8 discriminators are used for training: multi-period discriminators (MPD) [23] with periods $\{2, 3, 5, 7, 11\}$, and multi-band multi-scale complex STFT discriminators (MBMS-STFT) [26, 46, 9] with window sizes $\{512, 1024, 2048\}$, each using 5 bands. The sum of

discriminator outputs is used in a least squares GAN objective, as in [26, 34], minimizing

$$\mathcal{L}_{LSGAN}(D) = \frac{1}{2}\mathbb{E}_x\left[(D(x)-1)^2\right] + \frac{1}{2}\mathbb{E}_x\left[(D(G(x))+1)^2\right],$$
$$\mathcal{L}_{adv} = \mathbb{E}_x\left[(D(G(x))-1)^2\right].$$

- **Feature matching loss** — $L1$ of the discriminator feature maps for real and generated audio, taking expectation over all layers and discriminators, as

$$\mathcal{L}_{FM}(G, D_k) = \mathbb{E}_x\left[\sum_{i=1}^{L}\frac{1}{N_i}\left\|D_k^{(i)}(x) - D_k^{(i)}(G(x))\right\|_1\right],$$

  where G and $D_k^{(i)}$ are the generator and the i-th layer of the k-th discriminator, N_i is the number of units in each layer, and x and $G(x)$ are true and generated audio samples. Feature matching is used at each intermediate layer of each discriminator.

The final loss is computed as the weighted average

$$\mathcal{L} = 15\mathcal{L}_{spec} + \mathcal{L}_{adv} + 2\mathcal{L}_{FM}.$$

The model uses 8 encoder/decoder layers with latent, main, and intermediate dimensions $d_l = 128$, $d_m = 2048$, and $d_i = 3072$, respectively. The STFT analysis/synthesis uses $n_{\text{FFT}} = 960$ and $n_{\text{hop}} = 480$ as FFT and hop sizes. With audio being sampled at 48 kHz, this results in an audio-to-latent compression ratio of 3.75.

2.4 Evaluation

We conduct an experimental evaluation of the Woosh-AE model on the following public and private datasets:

- **AudioCaps —** We use the test split of AudioCaps as public dataset for evaluation. It consists of 950 audio samples from AudioSet with crowd-sourced manual captions.
- **InternalSFX —** A subset of 5000 audio samples from an internal studio-quality audio library from a different audio library not used for training. This dataset has detailed but rather non-verbose descriptions that are used as captions. As opposed to AudioCaps, most of the content consists of original sounds for production, rather than composed audio scenes recorded in real environments. We downsample audio to 48 kHz for consistency with the other datasets and computational efficiency. This is referred to as our private evaluation data.

We compare Woosh-AE-Public and Woosh-AE-Private against the Stable Audio Open Variational autoencoder (SAO-VAE) [15], and the Descript (16kbps) [26] and Encodec (24kbps) [9] baselines, the two latter models being neural audio codecs. We compute the following metrics:

- **Log-Mel Distance** (MelDist) - L1 distance between ground-truth audio and reconstructed audio log mel spectra. 128 Mel bands were used.
- **Log-STFT Distance** (STFTDist) - L1 distance between ground-truth audio and reconstructed audio STFT magnitude spectra. A FFT size of 512 samples was used.
- **Scale Independent Signal-to-Distortion Ratio** (SI-SDR) - Ground truth audio norm to reconstruction error norm ratio, scaling the ground truth to match the reconstructed audio.

Table 1 shows results for the encoder/decoder evaluation. For both AudioCaps and InternalSFX datasets, Woosh-AE-Public and Woosh-AE-Private ourperform all other models by a large margin. For AudioCaps, Woosh-AE-Public obtains reconstruction metrics 85% lower in terms of Log-Mel distance (0.032 vs. 0.217) and 23% lower in Log-STFT distance (1.18vs. 1.55) compared to SAO-VAE. The numbers also show a large drop in SI-SDR for SAO-VAE, although a negative SI-SDR is also reported for this dataset in [15]. We suspect SI-SDR may not be a reliable metric for audio generated with a stochastic model such as SAO-VAE. The Descript neural audio codec, operating at 16kbps, obtains better metrics than SAO-VAE, which is not quantized, with an SI-SDR of 9.69dB, but still far from the 20.79dB for Woosh-AE-Public or 22.52dB for Woosh-AE-Private. Encodec, the only baseline working at 48kHz sample rate, performs poorly overall.

Table 1: Audio encoder/decoder model evaluation on sound datasets, both for the public AudioCaps-Test and the InternalSFX-Test datasets.

			AudioCaps-Test			InternalSFX-Test		
Model	**SampleRate**	**Params**	**MelDist**	**STFTDist**	**SI-SDR**	**MelDist**	**STFTDist**	**SI-SDR**
Descript	44.1kHz	76 M	.150	1.44	9.69	.081	.376	7.29
Encodec	48kHz	19 M	.266	2.00	7.79	.161	.784	5.22
SAO-VAE	44.1kHz	156 M	.217	1.55	-0.08	.121	.462	-2.92
Woosh-AE-Public	48kHz	221 M	**.032**	**1.18**	20.79	**.021**	**.272**	12.7
Woosh-AE-Private	48kHz	221 M	.074	1.25	**22.52**	.024	.288	**16.03**

For the InternalSFX test set, both Woosh-AE models still perform comparably close, even though no commercial sound data was used for training Woosh-AE-Public. Absolute SI-SDR for these models are still high at 12.7dB and 16.03dB, lower than for AudioCaps, suggesting InternalSFX a is more demanding dataset, e.g in terms of transients and high-frequency content. Compared to Woosh-AE-Private, SAO-VAE results in 82% and 41% lower Log-Mel (0.021 vs. 0.121) and Log-STFT (0.272 vs. 0.462) distance metrics, respectively.

The metrics above, although favorable to Woosh-AE need to be put into perspective. SAO-VAE uses a latent dimension of 64 at a frame rate of 21.5Hz, and the sample rate is 44.1kHz. Woosh-AE models use a latent dimension of 128 at a 5x faster frame rate of 100Hz, with audio being sampled at 48kHz. The compression ratio for SAO-VAE is 0.03 whereas it is 0.26, i.e. 9 times lower, for Woosh-AE. Woosh-AE is also the largest model with 221M parameters, followed by SAO-VAE with 156M, and far above Descript or Encodec.

3 Woosh-CLAP: Text Conditioning

3.1 Architecture

The Woosh-CLAP text conditioning model uses a text encoder trained for text-audio alignment following the contrastive language-audio pretraining (CLAP) [12] approach. CLAP uses text and audio encoder models that produce text and audio embeddings, respectively. During training with a multi-modal dataset, positive text-audio embedding pairs are pushed together and negative pairs are pulled apart. As a result, text and audio embeddings end up living on a the same vector space, so-called *shared space*. This space, where audio can be predicted from text, and vice-versa, turns out to be highly semantic.

Woosh-CLAP uses RoBERTa-Large [33] as the text encoder and PaSST [24] as the audio encoder. A linear head is used to project audio- and text-pooled embeddings to a shared space of dimension 1024. RoBERTa has 355 M parameters, 24 transformer layers, 16 attention heads, and a hidden dimension of 1024. PaSST uses patched spectrogram inputs and a similar architecture to vision transformers [11], with 86 M parameters, 12 transformer blocks, 12 attention heads, and a hidden dimension of 768. PaSST has some desirable properties, such as being able to handle long audios by aggregating per-chunk CLAP embeddings. In comparison, other CLAP flavors like MS-CLAP and LAION-CLAP [43] can only consider a few seconds of audio. The audio encoder is dropped after training, as it is not required for text embedding computation and text-to-audio generation.

3.2 Data

To train the Woosh-CLAP public model, we used the Freesound, AudioCaps, and WavCaps datasets, described in Section 2.2. The Woosh-CLAP Private model was trained using the same mix of internal studio-quality sound effect libraries as described in the same section. Compared to the public datasets, the captions provided by these internal datasets were clearly less verbose, keyword-like, and using specific technical vocabulary and abbreviations. We used a large language model (LLM) to generate alternative descriptions that were more verbose and closer to natural language.

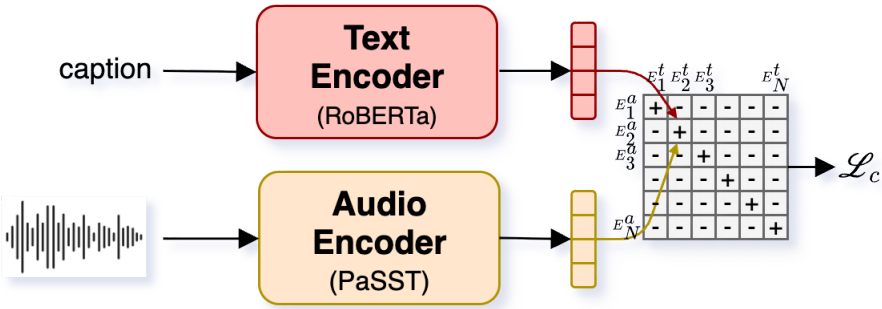

Figure 3: CLAP training block diagram for a positive pair of samples. Only the text encoder is used at inference time to condition the SFXFM generative model.

Pre-processing consisted of resampling the audio down to 32 kHz, the PaSST operating sampling rate, and training was done by taking random, non-segmented 10-second long chunks from the full audio sample.

3.3 Training

The official RoBERTa-Large and PaSST models, pretrained on multiple tasks using large amounts of heterogeneous text and audio data, served as a solid fine-tuning base. Linear text and audio heads were appended to the encoders as a projection from their respective embedding dimensions to the shared space dimension. Fine-tuning helps reduce text and audio domain shifts found in sound effect audio and in captions, in particular.

Assuming the i-th training sample pair having a text caption $\mathbf{X_i^t}$ and an audio sample $\mathbf{X_i^a}$, we project to the shared space by

$$E_i^t = \mathbf{W_t} \mathrm{f}(\mathbf{X_i^t}) + \mathbf{b}_t,$$
$$E_i^a = \mathbf{W_a} \mathrm{f}(\mathbf{X_i^a}) + \mathbf{b}_a.$$

The symmetric contrastive loss [37] is then computed as

$$L = \frac{1}{2N} \sum_{i=1}^{N} \left(\log \frac{\exp\left(E_i^a \cdot E_i^t / \tau\right)}{\sum_{j=1}^{N} \exp\left(E_i^a \cdot E_j^t / \tau\right)} + \log \frac{\exp\left(E_i^t \cdot E_i^a / \tau\right)}{\sum_{j=1}^{N} \exp\left(E_i^t \cdot E_j^a / \tau\right)} \right),$$

where N is the batch size and τ is a temperature value. The numerators are computed for the positive pairs, i.e., the diagonal elements in the cosine similarity matrix of Figure 3, and denominators are computed for all pairs, i.e., all cosine similarity elements.

We compute the contrastive loss on gathered text and audio embeddings over 4 GPUs, each with batch size 112, resulting in an effective batch size of $N = 448$ samples. Increasing the effective batch size did not have a positive effect on retrieval metrics. A fixed temperature of $\tau = 0.2$ was used.

3.4 Evaluation

We conduct the evaluation on the same public and private datasets used in Section 2.4 as they highlight the influence of the data choice on the reported metrics. We evaluated three CLAP models: 1) LAION-CLAP, using RoBERTa+HTS-AT encoders [33, 5], 2) Woosh-CLAP trained on public datasets, and 3) Woosh-CLAP trained on private studio-quality datasets. We assessed their performance in terms of recall in text-to-audio (T2A) and audio-to-text (A2T) retrieval tasks, and on the aforementioned AudioCaps and internal test datasets.

Table 2 shows different recall metrics for the three models. Considering the public test set, LAION-CLAP outperforms Woosh-CLAP-Public by up to 13% relative improvement (0.546 to 0.618), for T2A-R@10. Considering both models use similar architectures and training criterion, and LAION-CLAP using a non-negligible proportion of commercial training data, we attribute the observed performance gap to the training

Table 2: CLAP model evaluation on sound datasets. Recall at 5 (R@5) and recall at 10 (R@10) are given for text-to-audio (T2A) and audio-to-text (A2T) retrieval, both for the public AudioCaps-Test and the InternalSFX-Test datasets.

Model	**Params**	**AudioCaps-Test**				**InternalSFX-Test**			
		T2A		A2T		T2A		A2T	
		R@5	R@10	R@5	R@10	R@5	R@10	R@5	R@10
LAION-CLAP	355 M+31 M	**0.469**	**0.618**	**0.467**	**0.650**	0.28	0.188	0.215	0.328
Woosh-CLAP-Public	355 M+86 M	0.428	0.546	0.462	0.595	0.138	0.198	0.281	0.401
Woosh-CLAP-Private	355 M+86 M	0.095	0.167	0.153	0.227	**0.592**	**0.655**	**0.694**	**0.766**

dataset choices and the amount of training data. Woosh-CLAP-Private, trained exclusively on professional audio libraries compares poorly to both LAION-CLAP and Woosh-CLAP-Public models. However, when considering the internal dataset evaluation consisting of studio-quality sound effect data, we see a totally different picture: recall figures for LAION-CLAP and Woosh-CLAP-Public models lie far behind Woosh-CLAP-Private, the latter obtaining a relative improvement up to 248% higher over LAION-CLAP (0.655 vs. 0.188) in T2A-R@10. A similar trend is observed for A2T retrieval, where LAION-CLAP and Woosh-CLAP-Public also perform poorly, with Woosh-CLAP-Private obtaining over 2 times higher A2T-R@10 (0.766 vs. 0.328).

These results highlight the importance of using commercial audio libraries when targeting professional applications. In our experience, while in-the-wild audio can sometimes be of recording quality that is comparable to open datasets, its content can be structured very differently. On the other hand, public audio data tend to be real recordings, involving a mix of source sounds, making it difficult to disentangle each sound for production purposes. They are often also unintentionally noisy. The annotation style provided in professional libraries can also be highly mismatched to those provided in public datasets. We believe these factors explain the disparity observed in the reported evaluation results to a large extent.

4 Woosh-Flow: Text-to-Audio Generation

4.1 Architecture

The Woosh-Flow generative model is a latent diffusion model operating on audio-encoded latents, extracted using the Woosh-AE model described in Section 2. It is based on the FLUX Kontext architecture [27], using multimodal diffusion transformer blocks and trained with the flow matching objective [29]. The model uses a stack of multimodal transformer blocks, handling text and noise latent modalities, as shown in Figure 4. MultiStream modality blocks compute both self-attention and feed-forward network outputs on each modality sequence independently, and use rotary positional embeddings (RoPE) [41]. SingleStream modality blocks compute self-attention on a single sequence of time-concatenated modality sequences, also using RoPE embeddings. Figure 5 shows detailed diagrams for each of these blocks. SingleStream blocks attending to any time step across any modality sequence result in implicit attention computation across modalities, which can be thought of as indirectly conditioning the diffusion model.

The chosen architecture has a total of 12 multimodal transformer blocks, 6 MultiStream and 6 SingleStream. The main, intermediate, and RoPE dimensions are $d_m = 1024$, $d_i = 4096$, and $d_r = 112$, respectively.

4.2 Data

To train the Woosh-Flow-Public model, we use the Freesound, AudioCaps, and WavCaps datasets, described in Section 2.2. To train the Woosh-Flow-Private model we used the same mix of four internal studio-quality sound effect libraries as described in the same section. Pre-processing consisted of resampling the audio to 48 kHz. Audio samples were pre-segmented for non-silence using an energy-based detector, joining contiguous segments closer than 2 seconds. Random 5-second long chunks from a random segment were taken for training and testing.

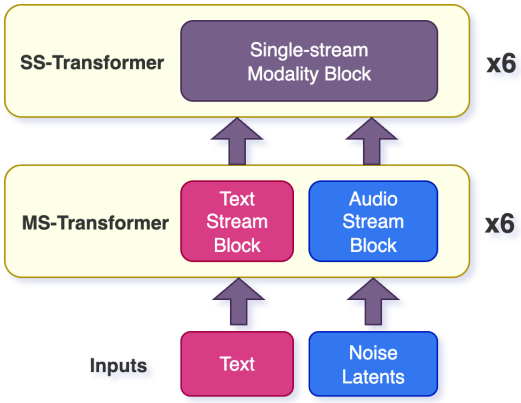

Figure 4: Multimodal transformer block stack in the Woosh-Flow diffusion model.

4.3 Training

In a first stage, the Woosh-Flow is trained using a flow matching (FM) objective [29, 30]. The model is meant to estimate a velocity field v_t that matches straight probability paths from data to noise. Assuming a linearly interpolated diffusion process of the form

$$x_t = (1-t) \cdot x_0 + t \cdot x_1 \,,$$

with x_0 and x_1 being data and noise samples, respectively, its time derivative

$$v_t = \frac{dx_t}{dt} = x_1 - x_0 \tag{1}$$

is used as target velocity for the FM L2 loss:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{x_0, x_1, t} \left\| u_\theta(x_t, t) - (x_1 - x_0) \right\|^2 \,. \tag{2}$$

The diffusion model network $u_\theta(x_t, t)$ is conditioned, as seen in the building blocks of Figure 5, on both a noisy sample x_t and its corresponding time step t and approximates the target velocity $x_1 - x_0$. By learning this velocity field, the trained model is used as a denoiser to estimate the original sample x_0 from any noisy intermediate x_t by solving the related differential equation $\frac{dx_t}{dt} = -u_\theta(x_t, t)$.

The criterion in Eq. 2 is used to pretrain the diffusion model for 300 epochs, using the Adam optimizer with 1000 warmup steps, 10^{-4} learning rate, and gradient clipping value at 1.0. t is sampled from a uniform distribution in the range $[0.001, 1]$. The model parameters are smoothed with an exponential moving average and a 0.999 decay. For text-conditioned diffusion, the token sequence of the second last layer of the Woosh-CLAP text encoder is used as conditioning signal.

At inference time, the learned dynamics via velocity estimation define a well-specified initial-value problem, for which arbitrary ordinary differential equation (ODE) solvers can be used, e.g., higher-order adaptive solvers such as Dormand–Prince (DOPRI5). In practice, a trade-off between reconstruction quality and number of function evaluations (NFEs) is desired.

4.4 Distillation

As a second training stage, the pretrained diffusion model is further distilled using the MeanFlow (MF) approach [17], allowing to reduce the NFEs at inference time from 100 to 8 with negligible audio quality loss. We call this model Woosh-DFlow.

Instead of the FM instantanous velocity target of Eq. 1, MeanFlow introduces the average velocity $u_{t,r}$, defined as

$$u_{t,r} = \frac{1}{t-r} \int_r^t v_\tau d\tau \,, \tag{3}$$

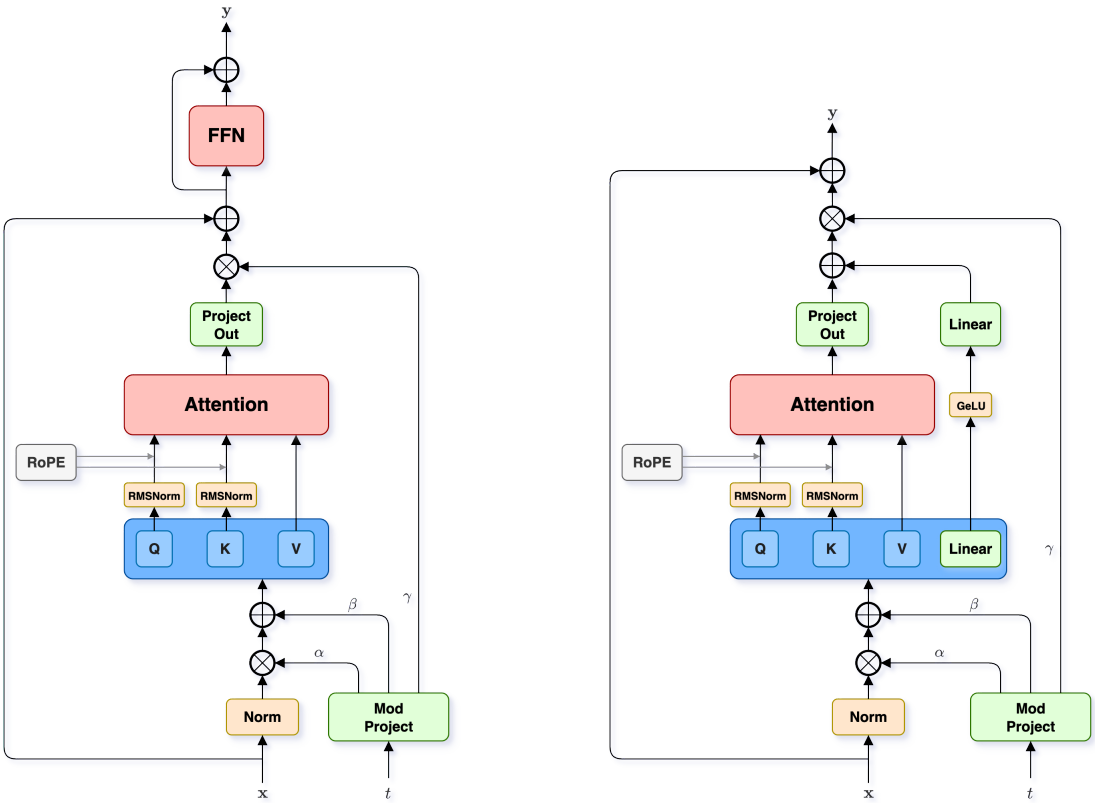

Figure 5: MultiStream transformer block diagram (left). Both self-attention and feed-forward network (FFN) outputs are computed independently for each modality sequence. SingleStream transformer block diagram (right). Self-attention is computed on the concatenation of modality sequences along the time dimension. Instead of an FFN block, non-linear features are in a parallel path and added after the attention project layer.

that is, the average of the velocities in the interval $[r, t]$, and it is used as the velocity field target in Eq. 2. Differentiating both sides of Eq. 3 yields the identity

$$u(x_t, t, r) = v(x_t, t) - (t - r) \cdot \frac{d}{dt} u(x_t, t, r) \,,$$

which, replaced as velocity target into the FM loss, yields the loss function

$$\mathcal{L}_{\text{MF}}(\theta) = \mathbb{E}_{x_0, x_1, t, r} \left\| u_\theta(x_t, t, r) - (x_1 - x_0) + (t - r) \cdot \frac{d}{dt} u_{\theta^-}(x_t, t, r) \right\|^2 . \tag{4}$$

Here, u_{θ^-} in the last term indicates that gradients are stopped to avoid higher order derivatives. The Jacobian vector product (JVP) operation is used to evaluate $\frac{d}{dt} u_{\theta^-}(x_t, t, r)$.

In our experience, training the diffusion model from scratch using Eq. 4 proved to be slow, with the JVP operation being computationally expensive, even when backing off to the FM criterion for a large proportion of training samples, as proposed in [17]. We switched to distillation mode, where the velocity target is the prediction by the teacher model $v_{\text{tgt}}(x_t, t)$ instead of $(x_1 - x_0)$, yielding the distillation loss

$$\mathcal{L}^{\text{d}}_{\text{MF}}(\theta) = \mathbb{E}_{x_0, x_1, t, r} \left\| u_\theta(x_t, t, r) - v_{\text{tgt}}(x_t, t) + (t - r) \cdot \frac{d}{dt} u_{\theta^-}(x_t, t, r) \right\|^2 . \tag{5}$$

Considering that Woosh-Flow is a text-conditioned model, the MeanFlow procedure can account for classifier-free guidance (CFG) [19] at training time. This enables the model to save the unconditional evaluation required for CFG at inference time, and thus half of the NFEs. For this purpose, the MeanFlow distillation loss in Eq. 5 is further modified to evaluate the teacher target velocity with the CFG already applied, i.e., $v'_{\text{tgt}}(x_t, c, t) = (1 - w) \cdot v_{\text{tgt}}(x_t, c, t) + w \cdot v_{\text{tgt}}(x_t, t)$, and taking the expectation over the distribution

Table 3: Woosh-Flow and Woosh-DFlow results on AudioCaps-Test and InternalSFX-Test. We provide the number latent diffusion model parameters (Params), the number of neural function evaluations at inference time (NFEs), and the FD, KL, and metrics described in Section 4.5. All models are conditioned on text.

Approach	**Params**	**NFEs**	**AudioCaps-Test**			**InternalSFX-Test**		
			FD ↓	KL ↓	CLAP ↑	FD ↓	KL ↓	CLAP ↑
SAO	1057 M	2×100	150.3	4.00	0.149			
Woosh-Flow-Public	337 M	≈2×70	**112.0**	**1.50**	**0.367**			
Woosh-Flow-Private	337 M	≈2×70	167.2	2.48	0.322			
Woosh-DFlow-Public	337 M	4						
Woosh-DFlow-Private	337 M	4						

of CFG weights of interest as well:

$$\mathcal{L}_{\text{MF}}^{\text{d,CFG}}(\theta) = \mathbb{E}_{x_0,x_1,t,r,w} \left\| u_\theta(x_t, c, t, r) - v'_{\text{tgt}}(x_t, c, t) + (t-r) \cdot \frac{d}{dt} u_{\theta^-}(x_t, t, r) \right\|^2 .$$

To further improve the distilled model, we also add an adversarial objective during distillation using a discriminator alongside the generator. Based on the latent adversarial diffusion distillation (LADD) approach [39], the discriminator learns to distinguish noisy ground truth (i.e., true samples), from noisy student-predicted (i.e., fake samples). The frozen teacher model is used as discriminator with the only addition of several convolutional heads that directly operate on the latent representations from different layers. The GAN hinge loss [28] is used in the discriminator together with adversarial losses. We adopt a similar approach to SANA [4] to compute partially denoised samples, except we directly denoise from t to $r < t$ as $x_r^{\text{fake}} = x_t - (t-r) \cdot \text{student}(x_t, t, r)$, instead of denoising from t to 0 and then re-noise to r in SANA. We provide a detailed pseudocode for this training process in Algorithm 1 in Appendix A.

We set the number of distillation fine-tuning epochs to 50, with a learning rate of $5 \cdot 10^{-6}$. To stabilize training in the first epochs, the MeanFlow joint (t, r) embeddings were fine-tuned for 1000 steps to match the t embeddings in the teacher network. CFG is uniformly sampled in the range [1,9] with a condition dropout rate of 0.1. The discriminator uses 4 convolutional heads. The adversarial loss weight is set to 0.5. For inference, a generic ODE solver, namely DOPRI5, is used with target absolute and relative tolerances of 10^{-3}.

4.5 Evaluation

We assess audio fidelity and semantic alignment for the Woosh-Flow and Woosh-DFlow models versus SAO, on the public and private datasets of Section 2.4, using the following metrics:

- **Audio quality and fidelity —** To measure the perceptual quality of the generated audio, we compute the Fréchet distance between the OpenL3 embeddings [8] of the generated and the ground truth audio samples (FD). Additionally, we use the Kullback-Leibler divergence between the PaSST embeddings [24] (or class probabilities) of the generated audio and the corresponding ground truth (KL), quantifying how well the model aligns acoustic and semantic content.
- **Cross-modal alignment —** We evaluate the alignment between the generated audio and the input conditioning modalities using the LAION-CLAP [42] score (CLAP), assessing text-to-audio relevance by computing the cosine similarity between the target text caption and the generated audio.

The inference setup for the three models was set as close as possible. We use a CFG scale of 7, 100 sampling steps for SAO, and a relative tolerance of 10^{-3} for Woosh-Flow, resulting in around 70 DOPRI5 integration steps. Note that, for both SAO and Woosh-Flow, CFG forces two function evaluations, conditional and unconditional, per step, resulting in around 200 NFEs for completing inference. For Woosh-DFlow, we use a CFG scale of 7 and a first-order Euler solver, with 4 and 8 sampling steps also corresponding to the total number of NFEs for completing inference. The t-step scheduler is uniform and we use a renoising strategy, introducing noise at every sampling step, with weights $[0, 0.5, 0.5, 0.3]$ and $[0, 0.5, 0.5, 0.5, 0.5, 0.3, 0.3, 0.3]$ for 4 and 8 steps, respectively.

Table 3 shows... [TBC]

5 Woosh-VFlow: Video-to-Audio Generation

5.1 Architecture

We extend the pre-trained Woosh-Flow framework to support video-to-audio generation by introducing a video conditioner module and adapting the transformer architecture for a third modality.

[TBC]

5.2 Data

To train the video-to-audio generation model, we utilize two large-scale audio-visual datasets: VGGSound and OGameData250k (see below). A key challenge in training on these datasets is the alignment between audio and text descriptions; therefore, we employ an audio-captioning pipeline using the Qwen3-Omni [44] audio-language model to enrich the training supervision.

- **VGGSound —** The VGGSound [3] dataset contains 183 k video clips for training, each of 10 s in duration. While extensive, the dataset presents challenges regarding data quality: the visual content is often noisy or loosely related to the audio (e.g., containing unrelated background music or speech), and the original labels are frequently inaccurate, often describing only a fraction of the auditory scene or isolated acoustic events. To address this, we generated synthetic captions for the dataset using Qwen3-Omni. We processed the audio in 5-second windows with a 2.5-second hop size to capture temporal evolution. During training, we employ a mixing strategy: we randomly select either the generated caption or the original dataset label with a probability of 0.5. Synthetic captions are only used during training.
- **OGameData250k —** We also incorporate the OGameData250k dataset, a subset of GameGen-X [2], consisting of 226 k video samples after removing out the test set. Following a setup similar to [38], we identified that the original annotations in this dataset primarily describe visual scenes rather than auditory events. Consequently, we discard the original labels entirely for audio conditioning. Instead, we exclusively train using synthetic captions generated via the same pipeline used for VGGSound (Qwen3-Omni on 5-second clips), ensuring the text conditioning accurately reflects the audio modality. We report our results on the OGameData250k test set, utilizing the identical test split defined in [38].

For evaluation, we also use the FoleyBench dataset [10], a high-quality benchmark designed for visually grounded audio generation. It consists of 5000 curated instances focusing on non-speech and non-music events. Unlike the large-scale training sets, FoleyBench provides cleaner, more strictly aligned audio-visual pairs, serving as our primary benchmark for assessing generation fidelity and alignment in a controlled setting.

5.3 Training

Visual features are extracted from input video sequences using the SynchFormer [21] encoder at a frame rate of 24 Hz. These embeddings are projected via a linear layer to match the diffusion transformer's hidden dimension, resulting in a sequence of video condition tokens. To accommodate variable video durations, we utilize a validity mask for the real video embeddings. Masked-out positions, representing the unconditioned parts of the sequence, are assigned to a fixed, non-trainable embedding initialized with zero values. Additionally, to match the temporal dynamics of the visual data, we compute positional encodings for the video tokens by resampling the RoPE frequencies to align with the video token rate.

To integrate these tokens, we modify the pre-trained multimodal transformer blocks. We introduce a dedicated video modality alongside the existing text and noise latent streams. Specifically, we augment the MultiStream blocks by adding learnable projections for query, key, and value , as well as a distinct FFN specific to the video modality.

Training is initialized from the Woosh-Flow checkpoint (Section 4). We employ a mixed training strategy, sampling a balanced distribution of approximately 50% video-audio pairs and 50% audio-only data. During training, we apply conditioner dropout with a probability of 0.1 randomly masking real video embeddings to improve robustness and facilitate CFG.

5.4 Distillation

[TBD]

5.5 Evaluation

To comprehensively assess the performance of our model, we employ a suite of metrics covering audio quality, semantic alignment, and audio-visual synchronization. On top of the FD, KL, and CLAP metrics used in Section 4.5, we also assess:

- **Cross-modal alignment —** We evaluate the alignment between the generated audio and the input conditioning modalities using ImageBind (IB) [18], which assesses the visual-auditory alignment. We compute the average cosine similarity between the audio embeddings and the embeddings of video frames sampled at 2-second intervals.
- **Synchronization analysis —** We utilize the pre-trained SynchFormer [21] model to predict the synchronicity between the video and generated audio, following the protocol in MMAudio [6]. We observed significant irregularities with this metric. Specifically, the pre-trained SynchFormer model frequently assigns lower synchronization scores to the ground truth video than to generated samples. Furthermore, we noted that MMAudio achieves exceptionally high scores; we hypothesize this is because the MMAudio architecture conditions each audio frame directly on interpolated SynchFormer embeddings, effectively optimizing for this specific metric by design. In our experiments on OGameData, while SynchFormer predicts a desynchronization larger than 900 ms for our model, a manual qualitative inspection of the samples revealed no perceptual sync issues. Consequently, we report this metric only to follow common practice, and with the caveat that it may not reliably reflect perceptual synchronization in this context.

We evaluate our method on the test sets of FoleyBench and OGameData250k (see Section 5.2). Across these datasets, we compare our performance against MMAudio-M [6], which serves as a parameter-comparable baseline. Results on the common VGGSound-Test [3] are also shown in Appendix B.

Table 4 shows... [TBC]

6 Conclusion

This report describes Sony AI's Woosh model release for high-quality sound effect generation, mainly addressing text-to-audio and video-to-audio tasks. Woosh models trained on public and private datasets were benchmarked against other popular open-source models, resulting in competitive performance when evaluated on a public dataset. Woosh models trained on thousands of hours of sound effect libraries largely outperformed all other models trained on public datasets, highlighting not only the domain shift present from publicly available to commercial sound effect datasets in terms of audio properties and textual annotations, but also the relevance of using the appropriate data when targeting professional applications.

References

[1] Andrea Agostinelli, Timo I. Denk, Zalán Borsos, Jesse Engel, Mauro Verzetti, Antoine Caillon, Qingqing Huang, Aren Jansen, Adam Roberts, Marco Tagliasacchi, Matt Sharifi, Neil Zeghidour, and Christian Frank. *MusicLM: Generating Music From Text*. 2023. arXiv: 2301.11325 [cs.SD]. URL: https://arxiv.org/abs/2301.11325.

Table 4: Woosh-VFlow results on FoleyBench (top) and OGameData-Test (bottom). We provide the number latent diffusion model parameters (Params), whether the model is text-conditioned or not (TC), the FD, KL, and CLAP metrics described in Section 4.5, and the task-specific IB and DeSync metrics described in Section 5.5.

Dataset	Approach	Params	TC	FD ↓	KL ↓	IB ↑	DeSync ↓	CLAP ↑
FoleyBench	Ground truth	-	✓	0.000	0.000	0.300	0.628	0.231
	MMAudio-M	621M	✓	30.480	1.983	**0.297**	**0.400**	0.292
	Woosh-VFlow	413M	✓	**24.136**	**1.774**	0.293	0.575	**0.325**
	Woosh-DVFlow	-	✓	28.168	1.857	0.276	0.624	0.319
	Woosh-DVFlow-c3	-	✓	24.951	1.838	0.274	0.662	0.315
	Woosh-DVFlow-V-c3	-	✓	24.422	1.834	0.278	0.647	0.303
	Woosh-DVFlow-V	-	✓	31.020	1.860	0.280	0.598	0.307
	MMAudio-M	621M	×	38.517	2.466	0.268	**0.439**	**0.195**
	Woosh-VFlow	413M	×	**34.382**	**2.401**	**0.269**	0.602	0.176
	Woosh-DVFlow	-	×	44.190	2.521	0.252	0.647	0.176
	Woosh-DVFlow-c3	-	×	40.598	2.520	0.246	0.689	0.169
	Woosh-DVFlow-V-c3	-	×	**31.745**	2.403	0.248	0.674	0.182
	Woosh-DVFlow-V	-	×	40.808	2.441	0.255	0.630	0.184
OGameData	Ground truth	-	✓	0.000	0.000	0.311	0.942	nan
	MMAudio-M	621M	✓	87.186	2.267	0.215	0.927	**0.240**
	Woosh-VFlow	413M	✓	**11.150**	**1.006**	**0.327**	**0.913**	0.202
	Woosh-DVFlow	-	✓	15.145	1.086	0.312	**0.896**	0.199
	Woosh-DVFlow-c3	-	✓	13.054	1.078	0.306	0.910	0.203
	Woosh-DVFlow-V-c3	-	✓	**10.859**	1.076	0.304	0.922	0.197
	Woosh-DVFlow-V	-	✓	14.825	1.120	0.308	0.915	0.196
	MMAudio-M	621M	×	86.746	2.758	0.207	0.926	0.107
	Woosh-VFlow	413M	×	**11.793**	**1.376**	**0.336**	**0.893**	**0.123**
	Woosh-DVFlow	-	×	18.661	1.471	0.321	**0.887**	0.130
	Woosh-DVFlow-c3	-	×	16.382	1.442	0.312	0.900	**0.132**
	Woosh-DVFlow-V-c3	-	×	12.239	1.435	0.313	0.913	0.126
	Woosh-DVFlow-V	-	×	16.035	1.493	0.321	0.910	0.127

[2] Haoxuan Che, Xuanhua He, Quande Liu, Cheng Jin, and Hao Chen. "GameGen-X: Interactive Open-world Game Video Generation". In: *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL: https://openreview.net/forum?id=8VG8tpPZhe.

[3] Honglie Chen, Weidi Xie, Andrea Vedaldi, and Andrew Zisserman. "Vggsound: A Large-Scale Audio-Visual Dataset". In: *2020 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2020, Barcelona, Spain, May 4-8, 2020*. IEEE, 2020, pp. 721–725. DOI: 10.1109/ICASSP40776.2020.9053174. URL: https://doi.org/10.1109/ICASSP40776.2020.9053174.

[4] Junsong Chen, Shuchen Xue, Yuyang Zhao, Jincheng Yu, Sayak Paul, Junyu Chen, Han Cai, Song Han, and Enze Xie. "Sana-sprint: One-step diffusion with continuous-time consistency distillation". In: *arXiv preprint* (2025). arXiv: arXiv:2503.09641 [cs.GR].

[5] Ke Chen, Xingjian Du, Bilei Zhu, Zejun Ma, Taylor Berg-Kirkpatrick, and Shlomo Dubnov. *HTS-AT: A Hierarchical Token-Semantic Audio Transformer for Sound Classification and Detection*. 2022. arXiv: 2202.00874 [cs.SD]. URL: https://arxiv.org/abs/2202.00874.

[6] Ho Kei Cheng, Masato Ishii, Akio Hayakawa, Takashi Shibuya, Alexander G. Schwing, and Yuki Mitsufuji. "MMAudio: Taming Multimodal Joint Training for High-Quality Video-to-Audio Synthesis". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2025, Nashville, TN, USA, June 11-15, 2025*. Computer Vision Foundation / IEEE, 2025, pp. 28901–28911. DOI: 10.1109/CVPR52734.2025.02691.

[7] Jade Copet, Felix Kreuk, Itai Gat, Tal Remez, David Kant, Gabriel Synnaeve, Yossi Adi, and Alexandre Défossez. *Simple and Controllable Music Generation*. 2024. arXiv: 2306.05284 [cs.SD]. URL: https://arxiv.org/abs/2306.05284.

[8] Jason Cramer, Ho-Hsiang Wu, Justin Salamon, and Juan Pablo Bello. "Look, Listen, and Learn More: Design Choices for Deep Audio Embeddings". In: *IEEE International Conference on Acoustics, Speech and Signal*

Processing, ICASSP 2019, Brighton, United Kingdom, May 12-17, 2019. IEEE, 2019, pp. 3852–3856. DOI: 10.1109/ICASSP.2019.8682475. URL: https://doi.org/10.1109/ICASSP.2019.8682475.

[9] Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi. *High Fidelity Neural Audio Compression*. 2022. arXiv: 2210.13438 [eess.AS]. URL: https://arxiv.org/abs/2210.13438.

[10] Satvik Dixit, Koichi Saito, Zhi-Wei Zhong, Yuki Mitsufuji, and Chris Donahue. "FoleyBench: A Benchmark For Video-to-Audio Models". In: *ArXiv* abs/2511.13219 (2025). URL: https://api.semanticscholar.org/CorpusID:283072409.

[11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV]. URL: https://arxiv.org/abs/2010.11929.

[12] Benjamin Elizalde, Soham Deshmukh, Mahmoud Al Ismail, and Huaming Wang. *CLAP: Learning Audio Concepts From Natural Language Supervision*. 2022. arXiv: 2206.04769 [cs.SD]. URL: https://arxiv.org/abs/2206.04769.

[13] Zach Evans, CJ Carr, Josiah Taylor, Scott H. Hawley, and Jordi Pons. *Fast Timing-Conditioned Latent Audio Diffusion*. 2024. arXiv: 2402.04825 [cs.SD]. URL: https://arxiv.org/abs/2402.04825.

[14] Zach Evans, Julian D. Parker, CJ Carr, Zack Zukowski, Josiah Taylor, and Jordi Pons. *Long-form music generation with latent diffusion*. 2024. arXiv: 2404.10301 [cs.SD]. URL: https://arxiv.org/abs/2404.10301.

[15] Zach Evans, Julian D. Parker, CJ Carr, Zack Zukowski, Josiah Taylor, and Jordi Pons. *Stable Audio Open*. 2024. arXiv: 2407.14358 [cs.SD]. URL: https://arxiv.org/abs/2407.14358.

[16] Jort F. Gemmeke, Daniel P. W. Ellis, Dylan Freedman, Aren Jansen, Wade Lawrence, R. Channing Moore, Manoj Plakal, and Marvin Ritter. "Audio Set: An ontology and human-labeled dataset for audio events". In: *Proc. IEEE ICASSP 2017*. New Orleans, LA, 2017.

[17] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J Zico Kolter, and Kaiming He. "Mean flows for one-step generative modeling". In: *arXiv preprint* (2025). arXiv: 2505.13447 [cs.LG].

[18] Rohit Girdhar, Alaaeldin El-Nouby, Zhuang Liu, Mannat Singh, Kalyan Vasudev Alwala, Armand Joulin, and Ishan Misra. "ImageBind One Embedding Space to Bind Them All". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*. IEEE, 2023, pp. 15180–15190. DOI: 10.1109/CVPR52729.2023.01457. URL: https://doi.org/10.1109/CVPR52729.2023.01457.

[19] Jonathan Ho and Tim Salimans. *Classifier-Free Diffusion Guidance*. 2022. arXiv: 2207.12598 [cs.LG]. URL: https://arxiv.org/abs/2207.12598.

[20] Chia-Yu Hung, Navonil Majumder, Zhifeng Kong, Ambuj Mehrish, Amir Ali Bagherzadeh, Chuan Li, Rafael Valle, Bryan Catanzaro, and Soujanya Poria. *TangoFlux: Super Fast and Faithful Text to Audio Generation with Flow Matching and Clap-Ranked Preference Optimization*. 2025. arXiv: 2412.21037 [cs.SD]. URL: https://arxiv.org/abs/2412.21037.

[21] Vladimir Iashin, Weidi Xie, Esa Rahtu, and Andrew Zisserman. *Synchformer: Efficient Synchronization from Sparse Cues*. 2024. arXiv: 2401.16423 [cs.CV]. URL: https://arxiv.org/abs/2401.16423.

[22] Chris Dongjoo Kim, Byeongchang Kim, Hyunmin Lee, and Gunhee Kim. "AudioCaps: Generating Captions for Audios in The Wild". In: *NAACL-HLT*. 2019.

[23] Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae. *HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis*. 2020. arXiv: 2010.05646 [cs.SD]. URL: https://arxiv.org/abs/2010.05646.

[24] Khaled Koutini, Jan Schlüter, Hamid Eghbal-zadeh, and Gerhard Widmer. "Efficient Training of Audio Transformers with Patchout". In: *23rd Annual Conference of the International Speech Communication Association, Interspeech 2022, Incheon, Korea, September 18-22, 2022*. Ed. by Hanseok Ko and John H. L. Hansen. ISCA, 2022, pp. 2753–2757. DOI: 10.21437/INTERSPEECH.2022-227. URL: https://doi.org/10.21437/Interspeech.2022-227.

[25] Felix Kreuk, Gabriel Synnaeve, Adam Polyak, Uriel Singer, Alexandre Défossez, Jade Copet, Devi Parikh, Yaniv Taigman, and Yossi Adi. *AudioGen: Textually Guided Audio Generation*. 2023. arXiv: 2209.15352 [cs.SD]. URL: https://arxiv.org/abs/2209.15352.

[26] Rithesh Kumar, Prem Seetharaman, Alejandro Luebs, Ishaan Kumar, and Kundan Kumar. *High-Fidelity Audio Compression with Improved RVQGAN*. 2023. arXiv: 2306.06546 [cs.SD]. URL: https://arxiv.org/abs/2306.06546.

[27] Black Forest Labs, Stephen Batifol, Andreas Blattmann, Frederic Boesel, Saksham Consul, Cyril Diagne, Tim Dockhorn, Jack English, Zion English, Patrick Esser, Sumith Kulal, Kyle Lacey, Yam Levi, Cheng Li, Dominik Lorenz, Jonas Müller, Dustin Podell, Robin Rombach, Harry Saini, Axel Sauer, and Luke Smith. *FLUX.1 Kontext: Flow Matching for In-Context Image Generation and Editing in Latent Space*. 2025. arXiv: 2506.15742 [cs.GR]. URL: https://arxiv.org/abs/2506.15742.

[28] Jae Hyun Lim and Jong Chul Ye. *Geometric GAN*. 2017. arXiv: 1705.02894 [stat.ML]. URL: https://arxiv.org/abs/1705.02894.

[29] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. *Flow Matching for Generative Modeling*. 2023. arXiv: 2210.02747 [cs.LG]. URL: https://arxiv.org/abs/2210.02747.

[30] Yaron Lipman, Marton Havasi, Peter Holderrieth, Neta Shaul, Matt Le, Brian Karrer, Ricky TQ Chen, David Lopez-Paz, Heli Ben-Hamu, and Itai Gat. "Flow Matching Guide and Code". In: *arXiv preprint* (2024). arXiv: 2412.06264 [cs.LG].

[31] Haohe Liu, Zehua Chen, Yi Yuan, Xinhao Mei, Xubo Liu, Danilo Mandic, Wenwu Wang, and Mark D. Plumbley. *AudioLDM: Text-to-Audio Generation with Latent Diffusion Models*. 2023. arXiv: 2301.12503 [cs.SD]. URL: https://arxiv.org/abs/2301.12503.

[32] Haohe Liu, Yi Yuan, Xubo Liu, Xinhao Mei, Qiuqiang Kong, Qiao Tian, Yuping Wang, Wenwu Wang, Yuxuan Wang, and Mark D. Plumbley. *AudioLDM 2: Learning Holistic Audio Generation with Self-supervised Pretraining*. 2024. arXiv: 2308.05734 [cs.SD]. URL: https://arxiv.org/abs/2308.05734.

[33] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. 2019. arXiv: 1907.11692 [cs.CL]. URL: https://arxiv.org/abs/1907.11692.

[34] Xudong Mao, Qing Li, Haoran Xie, Raymond Y. K. Lau, Zhen Wang, and Stephen Paul Smolley. *Least Squares Generative Adversarial Networks*. 2017. arXiv: 1611.04076 [cs.CV]. URL: https://arxiv.org/abs/1611.04076.

[35] Xinhao Mei, Chutong Meng, Haohe Liu, Qiuqiang Kong, Tom Ko, Chengqi Zhao, Mark D. Plumbley, Yuexian Zou, and Wenwu Wang. "WavCaps: A ChatGPT-Assisted Weakly-Labelled Audio Captioning Dataset for Audio-Language Multimodal Research". In: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 32 (2024), pp. 3339–3354. ISSN: 2329-9304. DOI: 10.1109/taslp.2024.3419446. URL: http://dx.doi.org/10.1109/TASLP.2024.3419446.

[36] Jordi Pons, Santiago Pascual, Giulio Cengarle, and Joan Serrà. "Upsampling Artifacts in Neural Audio Synthesis". In: *Proc. of the IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP)*. 2021, pp. 3005–3009. URL: https://arxiv.org/abs/2010.14356.

[37] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. arXiv: 2103.00020 [cs.CV]. URL: https://arxiv.org/abs/2103.00020.

[38] Koichi Saito, Julian Tanke, Christian Simon, Masato Ishii, Kazuki Shimada, Zachary Novack, Zhi-Wei Zhong, Akio Hayakawa, Takashi Shibuya, and Yuki Mitsufuji. "SoundReactor: Frame-level Online Video-to-Audio Generation". In: *ArXiv* abs/2510.02110 (2025). URL: https://api.semanticscholar.org/CorpusID:281725129.

[39] Axel Sauer, Frederic Boesel, Tim Dockhorn, Andreas Blattmann, Patrick Esser, and Robin Rombach. "Fast High-Resolution Image Synthesis with Latent Adversarial Diffusion Distillation". In: *SIGGRAPH Asia 2024 Conference Papers*. 2024, pp. 1–11. DOI: 10.1145/3680528.3687625.

[40] Hubert Siuzdak. *Vocos: Closing the gap between time-domain and Fourier-based neural vocoders for high-quality audio synthesis*. 2024. arXiv: 2306.00814 [cs.SD]. URL: https://arxiv.org/abs/2306.00814.

[41] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2023. arXiv: 2104.09864 [cs.CL]. URL: https://arxiv.org/abs/2104.09864.

[42] Yusong Wu, Ke Chen, Tianyu Zhang, Yuchen Hui, Taylor Berg-Kirkpatrick, and Shlomo Dubnov. "Large-Scale Contrastive Language-Audio Pretraining with Feature Fusion and Keyword-to-Caption Augmentation". In: *IEEE International Conference on Acoustics, Speech and Signal Processing ICASSP 2023, Rhodes Island, Greece, June 4-10, 2023*. IEEE, 2023, pp. 1–5. DOI: 10.1109/ICASSP49357.2023.10095969. URL: https://doi.org/10.1109/ICASSP49357.2023.10095969.

[43] Yusong Wu, Ke Chen, Tianyu Zhang, Yuchen Hui, Marianna Nezhurina, Taylor Berg-Kirkpatrick, and Shlomo Dubnov. *Large-scale Contrastive Language-Audio Pretraining with Feature Fusion and Keyword-to-Caption Augmentation*. 2024. arXiv: 2211.06687 [cs.SD]. URL: https://arxiv.org/abs/2211.06687.

[44] Jin Xu, Zhifang Guo, Hangrui Hu, Yunfei Chu, Xiong Wang, Jinzheng He, Yuxuan Wang, Xian Shi, Ting He, Xinfa Zhu, Yuanjun Lv, Yongqi Wang, Dake Guo, He Wang, Linhan Ma, Pei Zhang, Xinyu Zhang, Hongkun Hao, Zishan Guo, Baosong Yang, Bin Zhang, Ziyang Ma, Xipin Wei, Shuai Bai, Keqin Chen, Xuejing Liu, Peng Wang, Mingkun Yang, Dayiheng Liu, Xingzhang Ren, Bo Zheng, Rui Men, Fan Zhou, Bowen Yu, Jianxin Yang, Le Yu, Jingren Zhou, and Junyang Lin. "Qwen3-Omni Technical Report". In: *CoRR* abs/2509.17765 (2025). DOI: 10.48550/ARXIV.2509.17765. arXiv: 2509.17765. URL: https://doi.org/10.48550/arXiv.2509.17765.

[45] Junichi Yamagishi, Christophe Veaux, and Kirsten MacDonald. *CSTR VCTK Corpus: English Multi-speaker Corpus for CSTR Voice Cloning Toolkit (version 0.92)*. 2019. URL: https://datashare.ed.ac.uk/handle/10283/2950.

[46] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. *SoundStream: An End-to-End Neural Audio Codec*. 2021. arXiv: 2107.03312 [cs.SD]. URL: https://arxiv.org/abs/2107.03312.

Appendix A Distillation Pseudocode

The algorithm 1 provides detailed pseudocode for the Woosh-DFLow training process, using the MeanFlow criterion together with latent adversarial distillation (LADD).

Algorithm 1 Distillation Training with Adversarial Loss

repeat
  Sample data and noise: $x_0 \sim p_{\text{data}},\ x_1 \sim \mathcal{N}(0, I)$
  Train discriminator:
  if `training_step` > 5000 **then**
    Unfreeze discriminator
    Sample $(t, r) \sim$ `tr_scheduler`
    Build noisy input: $x_t \leftarrow (1-t) \cdot x_0 + t \cdot x_1$
    Student's prediction: $x_r^{\text{fake}} \leftarrow x_t - (t-r) \cdot \texttt{student}(x_t, t, r)$
    Ground truth: $x_r^{\text{true}} \leftarrow (1-r) \cdot x_0 + r \cdot x_1$
    Discriminator loss and backpropagation:

$$\mathcal{L}_{\text{disc}} \leftarrow \text{ReLU}\big[1 - \texttt{disc}(x_r^{\text{true}})\big] + \text{ReLU}\big[1 + \texttt{disc}(x_r^{\text{fake}}\texttt{.detach()})\big]$$
$$\mathcal{L}_{\text{disc}}\texttt{.backward()}$$

  end if
  Train generator:
  Freeze discriminator
  Sample $(t, r) \sim$ `tr_scheduler`
  Build noisy input: $x_t \leftarrow (1-t) \cdot x_0 + t \cdot x_1$
  Get distillation target: $v_{\text{tgt}} \leftarrow \texttt{teacher}(x_t, t)\texttt{.detach()}$
  Student's prediction:

$$u, \tfrac{du}{dt} \leftarrow \texttt{jvp}\big(\texttt{student}(x_t, t, r), (v_{\text{tgt}}, 1, 0)\big)$$

  Tangent:

$$g \leftarrow \big(u - v_{\text{tgt}} + (t-r) \cdot \tfrac{du}{dt}\big)\texttt{.detach().clip}(-1, 1)$$

  Mean Flow loss:

$$\mathcal{L}_{\text{MF}} \leftarrow \|u - u\texttt{.detach()} + g\|^2$$

  Adversarial loss:

$$x_r \leftarrow x_t - (t-r) \cdot u$$
$$\mathcal{L}_{\text{adv}} \leftarrow -\texttt{disc}(x_r)$$

  Generator loss and backpropagation:

$$\mathcal{L}_{\text{gen}} \leftarrow \mathcal{L}_{\text{MF}} + 0.5 \cdot \mathcal{L}_{\text{adv}}$$
$$\mathcal{L}_{\text{gen}}\texttt{.backward()}$$

until convergence

Appendix B Additional Results

Table 5: Woosh-VFlow results on VGG-Sound.

Approach	Params	TC	FD	KL	IB	DeSync	CLAP
Ground truth	-	✓	-0.000	0.000	0.331	0.630	0.198
MMAudio-M	621M	✓	30.539	**1.413**	**0.329**	**0.449**	0.224
Woosh-VFlow	413M	✓	**25.711**	1.691	0.282	0.633	**0.278**
Woosh-DVFlow-c3	-	✓	25.404	1.698	0.276	0.688	0.252
Woosh-DVFlow-V-c3	-	✓	25.469	1.722	0.278	0.673	0.235
MMAudio-M	621M	Recap	31.445	1.540	**0.309**	**0.448**	**0.166**
Woosh-VFlow	413M	Recap	**19.442**	**1.338**	0.270	0.620	0.161
Woosh-DVFlow-c3	-	Recap	20.836	1.441	0.260	0.672	0.153
Woosh-DVFlow-V-c3	-	Recap	22.343	1.472	0.259	0.675	0.150
MMAudio-M	621M	×	30.528	**1.846**	**0.320**	**0.436**	**0.162**
Woosh-VFlow	413M	×	**20.837**	2.118	0.286	0.577	0.136
Woosh-DVFlow-c3	-	×	21.883	2.186	0.268	0.655	0.124
Woosh-DVFlow-V-c3	-	×	24.336	2.192	0.266	0.655	0.128